

Noyau du système d'exploitation

Généralités

Le **noyau d'un système d'exploitation** (abrégé **noyau**, ou **kernel** en anglais, de l'allemand **kern**), est la partie fondamentale de certains systèmes d'exploitation.

Le noyau est un programme unique responsable de la communication entre le matériel et le logiciel. En tant que partie du système d'exploitation, le noyau fournit des mécanismes d'abstraction du matériel, notamment de la mémoire, du (ou des) processeur(s), et des échanges d'informations entre logiciels et périphériques matériels. Le noyau autorise aussi diverses abstractions logicielles et facilite la communication entre les processus. Les noyaux fournissent également des modèles de pilotes et des pilotes pour le matériel.

La majorité des systèmes d'exploitation est construite autour de la notion de noyau. L'existence d'un noyau, c'est-à-dire d'un programme unique responsable de la communication entre le matériel et le logiciel, résulte de compromis complexes portant sur des questions de performance, de sécurité et d'architecture des processeurs.

Le noyau offre ses fonctionnalités (l'accès aux ressources qu'il gère) au travers des appels système. Il transmet ou interprète les informations du matériel via des interruptions. C'est ce que l'on appelle les entrées et sorties.

Diverses abstractions de la notion d'*application* sont fournies par le noyau aux développeurs. La plus courante est celle de processus (ou tâche). Le noyau du système d'exploitation n'est en lui-même pas une tâche, mais un ensemble de routines pouvant être appelées par les différents processus pour effectuer des opérations requérant un certain niveau de privilèges. Les flots d'exécution dans le noyau sont des continuations des flots d'exécution des processus utilisateurs bloqués lorsqu'ils effectuent des appels systèmes. En général, un processus bloqué ne consomme pas de temps processeur, il est réveillé par le processus système lorsque celui-ci se termine.

Un processeur est capable d'exécuter un seul processus, un multiprocesseur est capable de gérer autant de processus qu'il a de processeurs. Pour pallier cet inconvénient majeur, les noyaux multitâches permettent l'exécution de plusieurs processus sur un processeur, en partageant le temps du processeur entre les processus.

Lorsque plusieurs tâches doivent être exécutées de manière parallèle, un noyau multitâche s'appuie sur les notions de :

- commutation de contexte ;
- ordonnancement ;
- temps partagé.

Les entrées et les sorties font l'objet d'un traitement spécifique par l'ordonnanceur.

Systèmes à noyaux restreints

Il existe de nombreux noyaux aux fonctionnalités restreintes tels que les micro-noyaux, les systèmes sans noyau (MS-DOS, CP/M) ou les exo-noyaux.

Ces systèmes sont généralement adaptés à des applications très ciblées mais posent des problèmes variés (de sécurité avec MS-DOS, de performances avec HURD ou QNX). La plupart d'entre eux sont actuellement inadaptés pour une utilisation généraliste, dans des serveurs ou ordinateurs personnels.

Fonctions généralement remplies par un noyau

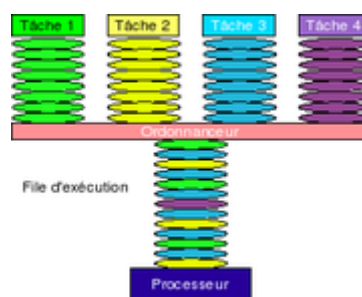
Les noyaux ont comme fonctions de base d'assurer le chargement et l'exécution des processus, de gérer les entrées/sorties et de proposer une interface entre l'espace noyau et les programmes de l'espace utilisateur.

À de rares exceptions, les noyaux ne sont pas limités à leurs fonctionnalités de base. On trouve généralement dans les noyaux les fonctions des micro-noyaux : un gestionnaire de mémoire et un ordonnanceur, ainsi que des fonctions de communication inter-processus.

En dehors de fonctions précédemment listées, de nombreux noyaux fournissent également des fonctions moins fondamentales telles que la gestion des systèmes de fichiers ; plusieurs ordonnanceurs spécialisés (batch, temps réel, entrées/sorties, etc.) ; des notions de processus étendues telles que les processus légers ; des supports réseaux (TCP/IP, PPP, pare-feu, etc.) ; des services réseau (NFS, etc.). Enfin, la plupart des noyaux fournissent également des modèles de pilotes et des pilotes pour le matériel.

En dehors des fonctionnalités de base, l'ensemble des fonctions des points suivants (y compris les pilotes matériels, les fonctions réseaux et systèmes de fichiers ou les services) n'est pas nécessairement fourni par un noyau de système d'exploitation. Ces fonctions du système d'exploitation peuvent être implantées tant dans l'espace utilisateur que dans le noyau lui-même. Leur implantation dans le noyau est faite dans l'unique but d'augmenter les performances. En effet, suivant la conception du noyau, la même fonction appelée depuis l'espace utilisateur ou l'espace noyau a un coût temporel notablement différent. Si cet appel de fonction est fréquent, il peut s'avérer utile d'intégrer ces fonctions au noyau pour augmenter les performances.

Ordonnanceur



4 tâches ordonnancées. La tâche 3 est en priorité haute, la tâche 4 est en priorité faible. *Ce diagramme est explicatif, en pratique les instructions ordonnées sont directement exécutées*

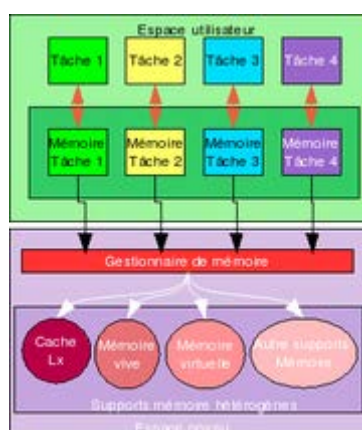
L'ordonnanceur d'un système d'exploitation n'a de sens qu'en système multitâche. Il gère l'ordre dans lequel les instructions de *différentes* tâches sont exécutées et est responsable de la sauvegarde et de la restauration du contexte des tâches (ce contexte est constitué des registres processeurs), appelée également commutation de contexte.

L'algorithme d'ordonnancement détermine quelle tâche doit s'exécuter en priorité et sur quel processeur. Cet algorithme doit permettre d'utiliser efficacement les ressources de la machine.

L'ordonnancement peut être de type « coopératif » : les tâches doivent être écrites de manière à coopérer les unes avec les autres et ainsi accepter leur suspension pour l'exécution d'une autre tâche. L'ordonnancement peut être également de type préemptif : l'ordonnanceur a la responsabilité de l'interruption des tâches et du choix de la prochaine à exécuter. Certains noyaux sont eux-mêmes préemptifs : l'ordonnanceur peut interrompre le noyau lui-même pour faire place à une activité (typiquement, toujours dans le noyau) de priorité plus élevée.

Gestionnaire de mémoire

Le gestionnaire de mémoire est le sous-ensemble du système d'exploitation qui permet de gérer la mémoire de l'ordinateur. Sa tâche la plus basique est d'allouer de la mémoire à des processus lorsqu'ils en ont besoin. Cette mémoire allouée est par défaut propre au processus qui en fait la demande.



Sur les noyaux récents, le gestionnaire de mémoire masque la localisation physique de la mémoire (en mémoire vive ou sur disque dur, dans l'espace de mémoire paginée) et présente au programme une *mémoire globale* uniforme dite mémoire virtuelle. Ainsi, tout processus croit manipuler une mémoire qui a les propriétés suivantes :

- la mémoire peut être étendue jusqu'aux capacités théoriques de la machine;
- la mémoire est privée (protégée), un processus ne peut pas accéder à la mémoire d'un autre processus (sauf allocations et autorisations spécifiques).

L'intérêt de ne pas indiquer au processus l'emplacement physique des données est de permettre au gestionnaire de mémoire de placer et déplacer à sa convenance les données en

mémoire, sans affecter les processus. Ces données peuvent notamment être fragmentées dans la mémoire vive lorsqu'un processus demande un bloc de mémoire d'une taille supérieure au plus grand bloc physique libre. Le contenu de la mémoire peut aussi être migré. Cette migration est faite sur les différents supports mémoires tels que dans la mémoire physique (plus ou moins proche du processeur), dans la mémoire paginée, dans la mémoire accessible par réseaux (grappe de calcul).

Appels système

Les appels système sont des fonctions :

- appelées depuis un programme de l'espace utilisateur ;
- dont l'exécution (le traitement) est effectuée dans l'espace noyau ;
- dont le retour est effectué dans le programme appelant dans l'espace utilisateur.

En plus d'un changement de mode d'exécution, l'appel système suppose au moins deux commutations de contextes :

1. Contexte du programme appelant ;
 - o changement de contexte ;
2. Contexte du noyau ;
 - o changement de contexte ;
3. Contexte du programme appelant.

Le coût d'un appel système est nettement plus élevé qu'un simple appel de fonction intra-processus : alors qu'un appel de fonction ne suppose que quelques instructions primitives (chargement et exécution d'une zone mémoire), le coût d'un appel système se compte en milliers ou dizaines de milliers d'instructions primitives, générant à la fois une charge et des délais d'exécution supplémentaires. Pour ces raisons, les fonctions qui sont utilisées de manière intense sont déplacées dans l'espace noyau. Les programmes utilisateurs font alors un nombre restreint d'appels système de haut niveau. Les nombreuses interactions de bas niveau générées par ces appels système sont effectuées dans l'espace noyau. Cela concerne notamment les pilotes de périphériques.

Les entrées/sorties font également l'objet d'un traitement par l'ordonnanceur.

Gestion du matériel

La gestion du matériel se fait par l'intermédiaire de pilotes de périphériques. Les pilotes sont des petits logiciels légers dédiés à un matériel donné qui permettent de faire communiquer ce matériel. En raison du très grand nombre d'accès à certains matériels (disques durs par exemple), certains pilotes sont très sollicités. Ils sont généralement inclus dans l'espace noyau et communiquent avec l'espace utilisateur via les appels système.

Cependant, indépendamment de l'architecture, de nombreux périphériques lents (certains appareils photographiques numériques, outils sur liaison série, etc.) sont/peuvent être pilotés depuis l'espace utilisateur, le noyau intervenant au minimum.

Il existe des couches d'abstraction de matériel (HAL) qui présentent la même interface à l'espace utilisateur et simplifient ainsi le travail des développeurs d'applications. Dans les systèmes de type UNIX, l'abstraction utilisée est le système de fichiers : les primitives *open*,

close, *read* et *write* sont présentées à l'espace utilisateur pour manipuler toutes sortes de périphériques. On parle dans ce cas de système de fichiers *synthétique*.

Différents types de noyaux

Il existe toutes sortes de noyaux, plus ou moins spécialisés. Des noyaux spécifiques à une architecture, souvent monotâches, d'autres généralistes et souvent multitâches et multiutilisateurs. L'ensemble de ces noyaux peut être divisé en deux approches opposées d'architectures logicielles : les noyaux monolithiques et les micro-noyaux.

On considère généralement les noyaux monolithiques, de conception ancienne, comme obsolètes car difficiles à maintenir et moins « propres ». Le noyau Linux était déjà qualifié d'obsolète par Andrew Tanenbaum, dès sa création en 1991. Il ne croyait pas, à l'époque, pouvoir faire un noyau monolithique multiplate-forme et modulaire. La mise en place de micro-noyaux, qui consiste à déplacer l'essentiel des fonctions du noyau vers l'espace utilisateur, est très intéressante en théorie mais s'avère difficile en pratique. Ainsi les performances du noyau Linux (monolithique) sont supérieures à celles de ses concurrents (noyaux généralistes à micro-noyaux), sans compter qu'il fut finalement porté sur de très nombreuses plates-formes et qu'il est modulaire depuis 1995.

Pour ces raisons de performance, les systèmes généralistes basés sur une technologie à micro-noyau, tels que Windows et Mac OS X, n'ont pas un « vrai » micro-noyau enrichi. Ils utilisent un micro-noyau hybride : certaines fonctionnalités qui devraient exister sous forme de mini-serveurs se retrouvent intégrées dans leur micro-noyau, utilisant le même espace d'adressage. Pour Mac OS X, cela forme XNU : le noyau monolithique BSD (Berkeley Software Distribution) fonctionne en tant que service de Mach et ce dernier inclut du code BSD dans son propre espace d'adressage afin de réduire les latences.

Ainsi, les deux approches d'architectures de noyaux, les micro-noyaux et les noyaux monolithiques, considérées comme diamétralement différentes en termes de conception, se rejoignent quasiment en pratique par les micro-noyaux hybrides et les noyaux monolithiques modulaires.

Noyaux monolithiques non modulaires



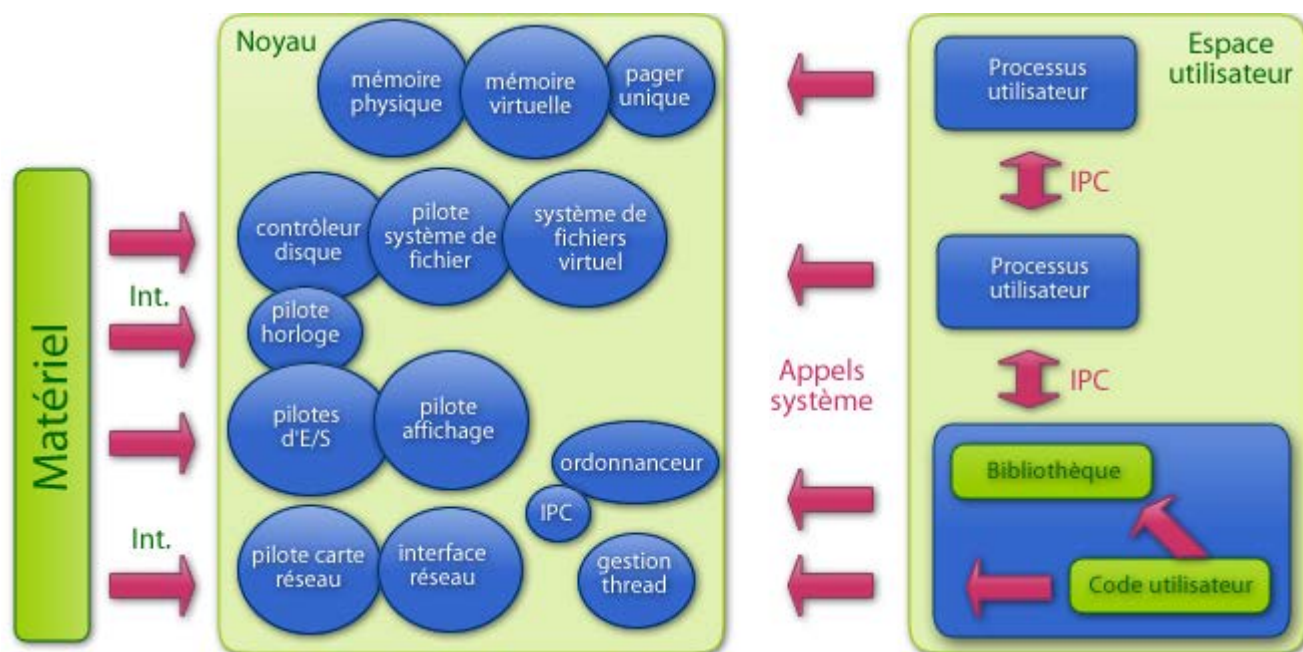
Architecture monolithique

Certains systèmes d'exploitation, comme d'anciennes versions de Linux, certains BSD ou certains vieux Unix ont un noyau monolithique. C'est-à-dire que l'ensemble des fonctions du

système et des pilotes sont regroupés dans un seul bloc de code et un seul bloc binaire généré à la compilation.

De par la simplicité de leur concept mais également de leur excellente vitesse d'exécution, les noyaux monolithiques ont été les premiers à être développés et mis en œuvre. Cependant, au fur et à mesure de leurs développements, le code de ces noyaux monolithiques a augmenté en taille et il s'est avéré difficile de les maintenir. Le support par les architectures monolithiques des chargements à chaud ou dynamiques implique une augmentation du nombre de pilotes matériels compilés dans le noyau, et par suite, une augmentation de la taille de l'empreinte mémoire des noyaux. Celle-ci devint rapidement inacceptable. Les multiples dépendances créées entre les différentes fonctions du noyau empêchaient la relecture et la compréhension du code. L'évolution du code s'est faite en parallèle à l'évolution du matériel, et des problèmes de portage ont alors été mis en évidence sur les noyaux monolithiques.

En réalité les problèmes de la portabilité de code se sont révélés avec le temps indépendants de la problématique de la technologie des noyaux. Pour preuve, NetBSD est un noyau monolithique et est porté sur un très grand nombre d'architectures, alors que des noyaux tels que HURD ou celui de Windows XP utilisent des micro-noyaux censés faciliter le portage mais n'existent que pour quelques architectures.



Architecture d'un noyau monolithique

Noyaux monolithiques modulaires

Pour répondre aux problèmes des noyaux monolithiques, ces derniers sont devenus modulaires. Dans ce type de noyau, seules les parties fondamentales du système sont regroupées dans un bloc de code unique (monolithique). Les autres fonctions, comme les pilotes matériels, sont regroupées en différents modules qui peuvent être séparés tant du point de vue du code que du point de vue binaire.

La très grande majorité des systèmes actuels utilise cette technologie : Linux, la plupart des BSD ou Solaris. Par exemple avec le noyau Linux, certaines parties peuvent être non

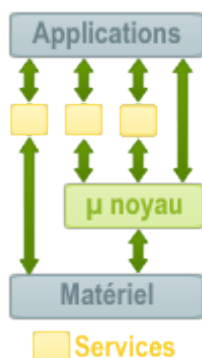
compilées ou compilées en tant que modules chargeables directement dans le noyau. La modularité du noyau permet le chargement à la demande de fonctionnalités et augmente les possibilités de configuration. Ainsi les systèmes de fichiers peuvent être chargés de manière indépendante, un pilote de périphérique changé, etc. Les distributions Linux, par exemple, tirent profit des modules chargeables lors de l'installation. L'ensemble des pilotes matériels sont compilés en tant que modules. Le noyau peut alors supporter l'immense variété de matériel trouvé dans les compatibles PC. Après l'installation, lors du démarrage du système, seuls les pilotes correspondants au matériel *effectivement* présent dans la machine sont chargés en mémoire vive. La mémoire est économisée.

Les noyaux monolithiques modulaires conservent les principaux atouts des noyaux monolithiques purs dont ils sont issus. Ainsi, la facilité de conception et de développement est globalement maintenue et la vitesse d'exécution reste excellente. L'utilisation de modules implique le découpage du code source du noyau en blocs indépendants. Ces blocs améliorent l'organisation et la clarté du code source et en facilitent également la maintenance.

Les noyaux monolithiques modulaires conservent également un important défaut des noyaux monolithiques purs : une erreur dans un module met en danger la stabilité de tout le système. Les tests et certifications de ces composants doivent être plus poussés.

D'un point de vue théorique, le grand nombre de lignes de code exécutées en mode noyau engendre des problèmes de portabilité. La pratique contredit largement la théorie et les noyaux modulaires sont aujourd'hui les plus portés.

Systèmes à micro-noyaux



Architecture d'un système à micro-noyau

Les limitations des noyaux monolithiques ont amené à une approche radicalement différente de la notion de noyau : les systèmes à micro-noyaux.

Les systèmes à micro-noyaux cherchent à minimiser les fonctionnalités dépendantes du noyau en plaçant la plus grande partie des services du système d'exploitation à l'extérieur de ce noyau, c'est-à-dire dans l'espace utilisateur. Ces fonctionnalités sont alors fournies par de petits serveurs indépendants possédant souvent leur propre espace d'adressage.

Un petit nombre de fonctions fondamentales est conservé dans un noyau minimaliste appelé « micro-noyau ». L'ensemble des fonctionnalités habituellement proposées par les noyaux monolithiques est alors assuré par les services déplacés en espace utilisateur et par ce micro-noyau. Cet ensemble logiciel est appelé « micro-noyau enrichi ».

Ce principe a de grands avantages théoriques : en éloignant les services « à risque » des parties critiques du système d'exploitation regroupées dans le noyau, il permet de gagner en robustesse et en fiabilité, tout en facilitant la maintenance et l'évolutivité. En revanche, les mécanismes de communication (IPC) qui deviennent fondamentaux pour assurer le passage de messages entre les serveurs, sont très lourds et peuvent limiter les performances.

Avantages et inconvénients d'un système à micro-noyau

Les avantages théoriques des systèmes à micro-noyaux sont la conséquence de l'utilisation du mode protégé par les services qui accompagnent le micro-noyau. En effet, en plaçant les services dans l'espace utilisateur, ceux-ci bénéficient de la protection de la mémoire. La stabilité de l'ensemble en est améliorée : une erreur d'un service en mode protégé a peu de conséquences sur la stabilité de l'ensemble de la machine.

De plus, en réduisant les possibilités pour les services de pouvoir intervenir directement sur le matériel, la sécurité du système est renforcée. Le système gagne également en possibilités de configuration. Ainsi, seuls les services utiles doivent être réellement lancés au démarrage. Les interdépendances entre les différents serveurs sont faibles. L'ajout ou le retrait d'un service ne perturbe pas l'ensemble du système. La complexité de l'ensemble est réduite.

Le développement d'un système à micro-noyau se trouve également simplifié en tirant parti à la fois de la protection de la mémoire et de la faible interdépendance entre les services. Les erreurs provoquées par les applications en mode utilisateur sont traitées plus simplement que dans le mode noyau et ne mettent pas en péril la stabilité globale du système. L'intervention sur une fonctionnalité défectueuse consiste à arrêter l'ancien service puis à lancer le nouveau, sans devoir redémarrer toute la machine.

Les micro-noyaux ont un autre avantage : ils sont beaucoup plus compacts que les noyaux monolithiques. 6 millions de lignes de code pour le noyau Linux 2.6.0 contre en général moins de 50 000 lignes pour les micro-noyaux. La maintenance du code exécuté en mode noyau est donc simplifiée. Le nombre réduit de lignes de code peut augmenter la portabilité du système.

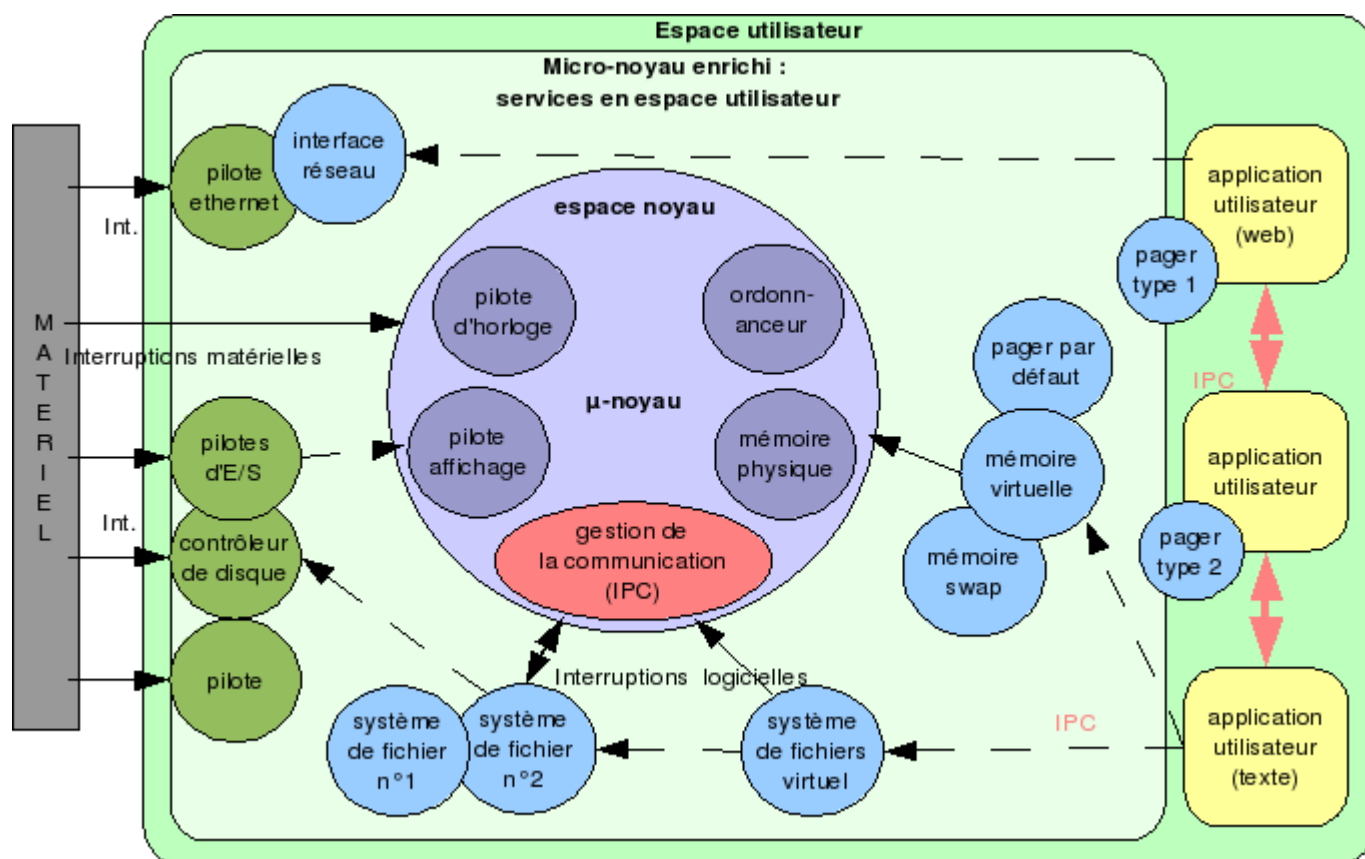
Les premiers micro-noyaux (comme Mach) n'ont pas tout de suite atteint ces avantages théoriques. L'utilisation de nombreux services dans l'espace utilisateur engendre les deux problèmes suivants :

1. La plupart des services sont à l'extérieur du noyau et génèrent un très grand nombre d'appels système ;
2. Les interfaces de communication entre les services (IPC) sont complexes et trop lourdes en temps de traitement.

Le grand nombre d'appels système et la communication sous-jacente sont un défaut inhérent à la conception des micro-noyaux. Dans L4, il a été résolu en plaçant encore plus de services en espace utilisateur. La rapidité de traitement des IPC a pu être améliorée en simplifiant les communications au maximum, par exemple en supprimant toute vérification des permissions, laissant ce soin aux serveurs externes.

Ces modifications radicales ont permis d'obtenir de bonnes performances mais elles ne doivent pas faire oublier qu'un micro-noyau doit être accompagné d'un grand nombre de services pour fournir des fonctionnalités équivalentes à celles des noyaux monolithiques. De

plus, la grande liberté dont disposent les services au niveau de la sécurité et de la gestion de la mémoire accroît la difficulté et le temps de leur développement (ils doivent fournir leurs propres interfaces).

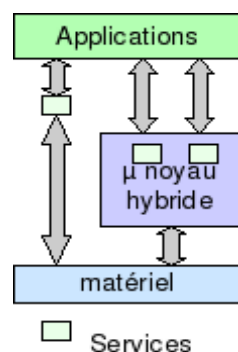


Architecture d'un micro-noyau enrichi par des services (micro-noyau enrichi)

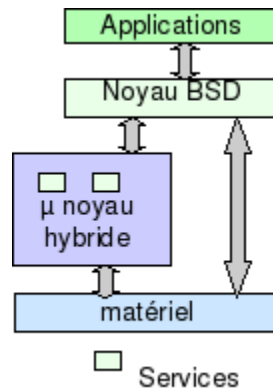
Exemple d'associations micro-noyaux - noyaux enrichis - système d'exploitation

Micro-noyau	Micro-noyau enrichi	Systèmes d'exploitation associés
L4	HURD	GNU/HURD
Mach	XNU	Darwin

Noyaux hybrides



Architecture hybride



Architecture hybride : XNU

La dénomination de « noyaux hybrides » désigne principalement des noyaux qui reprennent des concepts à la fois des noyaux monolithiques et des micro-noyaux, pour combiner les avantages des deux.

Lorsqu'au début des années 1990 les développeurs et concepteurs se sont aperçus des faiblesses des premiers micro-noyaux, certains réintégrèrent diverses fonctionnalités non fondamentales dans le noyau, pour gagner en performance. Les micro-noyaux « purs » semblaient condamnés à l'échec.

Alors que la philosophie générale des systèmes à micro-noyaux est maintenue (seules les fonctions fondamentales sont dans l'espace noyau), certaines fonctions non critiques, mais très génératrices d'appels système, sont réintégrées dans l'espace noyau. Ce compromis permet d'améliorer considérablement les performances en conservant de nombreuses propriétés des systèmes à micro-noyaux. Un exemple de ce type de noyau hybride est le noyau XNU de Mac OS X. Il est basé sur le micro-noyau Mach 3.0 mais qui inclut du code du noyau monolithique BSD au sein de l'espace noyau.

Cette dénomination est également utilisée pour désigner d'autres types de noyaux, notamment les noyaux monolithiques sur micro-noyaux (temps réel ou non) tels que L4Linux (Linux sur L4), MkLinux (le noyau Linux sur Mach), Adeos, RTLinux et RTAI.

Plus rarement, on peut rencontrer le terme « noyau hybride » pour remplacer improprement « noyau monolithique modulaire » ou « micro-noyau enrichi ».

Exo-noyaux

Étymologiquement, 'exo' signifie en grec 'hors de'. Un exo-noyau est donc un système d'exploitation fonctionnant en espace utilisateur (en 'user-space', au lieu du 'kernel-space' dans le cas des autres noyaux). Les fonctions et services du système d'exploitation sont assurés par de petits modules qui, selon les approches techniques, sont des bibliothèques dynamiques (MIT, LibOSes) ou des démons (IntraServices).

Méta-noyaux

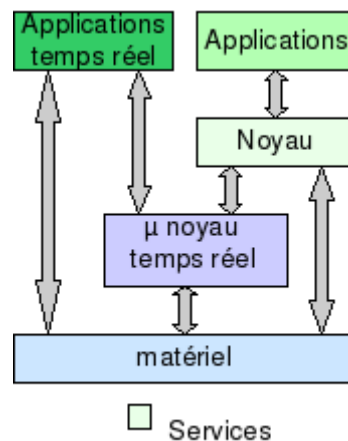
Un « méta-noyau » est un ensemble de logiciels qui vise à appliquer la notion de noyau informatique au niveau d'un réseau informatique, en créant une unique couche de gestion des périphériques au niveau d'un réseau.

De cette manière, les logiciels peuvent être déployés et utilisés sur le réseau informatique comme s'il s'agissait d'une machine unique, et l'ensemble des logiciels fonctionnant sur cette plate-forme peuvent se partager les ressources de manière intégrée, comme elle le ferait sur un noyau simple.

Un méta système doit également permettre la personnalisation, la gestion des permissions ainsi que l'utilisation d'informations dépendant de la localisation.

Cette notion rejoint les notions de grappe de calcul, de machine virtuelle, de serveur d'application et de CORBA.

Noyaux temps réel



Une possibilité d'architecture de noyau temps réel hybride

Les noyaux temps réel sont fonctionnellement spécialisés. Ce sont des noyaux généralement assez légers qui ont pour fonction de base stricte de garantir les temps d'exécution des tâches. Il n'y a pas à proprement parler de notion de rapidité de traitement ou de réactivité dans les noyaux temps réel, cette notion est plutôt implicite à la garantie des temps d'exécution en comparaison aux critères temporels de l'application industrielle (la réactivité d'un système de freinage ABS n'a pas les mêmes critères temporels que le remplissage d'une cuve de pétrole).

Très utilisés dans le monde de l'électronique embarquée, ils sont conçus pour tourner sur des plates-formes matérielles limitées en taille, puissance ou autonomie.

Les noyaux temps réel peuvent adopter en théorie n'importe quelle architecture précédemment listée. Ils fournissent souvent deux interfaces séparées, l'une spécialisée dans le temps réel et l'autre générique. Les applications temps réel font alors appel à la partie temps réel du noyau.

Une des architectures souvent retenue est un noyau hybride qui s'appuie sur la combinaison d'un micro-noyau temps réel spécialisé, allouant du temps d'exécution à un noyau de système d'exploitation non spécialisé. Le système d'exploitation non spécialisé fonctionne en tant que service du micro-noyau temps réel. Cette solution permet d'assurer le fonctionnement temps

réel des applications, tout en maintenant la compatibilité avec des environnements préexistants.

Par exemple, on peut avoir un micro-noyau temps réel allouant des ressources à un noyau non temps réel tel que Linux (RTLinux, RTAI) ou Windows. L'environnement GNU (resp. Windows) peut alors être exécuté à l'identique sur le noyau pour lequel il a été conçu, alors que les applications temps réel peuvent faire directement appel au micro-noyau temps réel pour garantir leurs délais d'exécutions.

VxWorks est un noyau propriétaire temps réel très implanté dans l'industrie bien que les systèmes à base de noyau Linux se déploient énormément et aient un succès grandissant via RTAI (RTLinux étant breveté).

Synthèse des principaux noyaux et de leurs architectures

Noyau	Noyau monolithique	Noyau monolithique modulaire	Micro-noyau	Micro-noyau enrichi	Noyau hybride	Temps réel	Exemples systèmes d'exploitation associés	de
AIX		✓ Oui			✗ Non	✓ Oui	AIX	
BeOS				✓ Oui	✓ Oui	✓ Oui	BeOS	
BSD 4.4		✓ Oui			✗ Non	✗ Non	BSD - Solaris 1	
Fiasco			✓ Oui			✓ Oui	GNU/L4Linux/Fiasco	
Irix		✓ Oui			✗ Non	✓ Oui	Irix	
L4			✓ Oui			✓ Oui	GNU/HURD ; GNU/L4linux	
Linux > 1.2		✓ Oui			✗ Non	✗ Non	GNU/Linux	
Mach			✓ Oui			✓ Oui	Mac OS X, Darwin, GNU/HURD, GNU/Mklinux	
NeXTStep				✓ Oui	✓ Oui	✗ Non	NeXTStep	
OS/2		✓ Oui			✗ Non	✗ Non	OS/2	
QNX				✓ Oui	✓ Oui	✓ Oui	QNX	
RT-OS360/75	✓ Oui				✗ Non	✓ Oui	IBM RTOS	
VxWorks				✓ Oui	✓ Oui	✓ Oui	Windows/VxWorks, BSD/VxWorks	
XNU				✓ Oui	✓ Oui	✓ Oui	Mac OS X, Darwin	